

Datové typy a vstup/výstup

Základy programování

Matouš Cejnek

U12110, FS, ČVUT v Praze

Co si dnes odnesete?

- Jak počítač ukládá informace pomocí bitů a bajtů?
- Jak se reprezentují celá a desetinná čísla?
- Jaké základní datové typy a kontejnery nabízí Python?
- Jak převádět hodnoty mezi typy?
- Jak program komunikuje s okolím přes vstup a výstup?

Na co navazujeme?

- Program pracuje s **vstupem**, provádí algoritmus a vytváří **výstup**.
- Proměnná dovoluje hodnotu pojmenovat a později ji použít.
- Dnes se ptáme, **jaký význam hodnota má a jak je uložena**.

Dnes neopakujeme řízení toku

Podmínky a cykly procvičíme na navazujícím semináři a cvičení.

Informace potřebuje reprezentaci

Data

Informace zapsaná v podobě, kterou lze uložit, přenášet a zpracovat.

- Stejný vzor bitů může podle dohody znamenat číslo, znak nebo barvu.
- **Datový typ** určuje možné hodnoty a povolené operace.
- Reprezentace propojuje abstraktní hodnotu s fyzickou pamětí.

Bit a bajt

- **Bit** je jedna binární číslice: 0 nebo 1.
- **Bajt (byte)** je běžně skupina osmi bitů.
- Osm bitů rozlišuje $2^8 = 256$ různých kombinací.

0 1 1 0 1 0 0 1

Jeden bajt; význam závisí na zvoleném typu a kódování.

Binární zápis celého čísla

00101101

$$\begin{aligned}00101101_2 &= 32 + 8 + 4 + 1 \\ &= 45_{10}\end{aligned}$$

- Každá pozice má váhu mocniny dvou.
- Nejpravější bit má váhu $2^0 = 1$.
- Přidání bitu zdvojnásobí počet možných stavů.

Celé číslo bez znaménka

Unsigned integer

Všech n bitů vyjadřuje velikost nezáporného celého čísla.

rozsah: 0 až $2^n - 1$

- 8 bitů: 0 až 255
- 16 bitů: 0 až 65 535
- Vhodné tam, kde záporná hodnota nedává smysl.

Jak uložit záporná čísla?

- Potřebujeme do stejného počtu bitů zahrnout kladné i záporné hodnoty.
- Samostatný „bit znaménka“ je intuitivní, ale přináší dvě reprezentace nuly.
- Moderní počítače obvykle používají **dvojkový doplněk**.

Důležitá myšlenka

Bity samy znaménko nemají. Znaménko vzniká až jejich interpretací podle dohodnuté reprezentace.

Celé číslo se znaménkem

Signed integer ve dvojkovém doplňku

Nejvyšší bit má zápornou váhu; ostatní bity mají obvyklé kladné váhy.

rozsah pro n bitů: -2^{n-1} až $2^{n-1} - 1$

- 8 bitů: -128 až 127
- Nula má jedinou reprezentaci.
- Stejně obvody mohou provádět sčítání kladných i záporných čísel.

Omezený počet bitů znamená omezený rozsah

- Výsledek mimo rozsah pevného typu způsobí **přetečení** (overflow).
- V některých prostředích hodnota „přeteče“ na druhý konec rozsahu.
- Jinde program ohlásí chybu nebo použije větší reprezentaci.

Python je v tomto neobvyklý

Pythonový `int` se podle potřeby zvětšuje; limitem je dostupná paměť, ne pevně 32 nebo 64 bitů.

Proč nestačí celá čísla?

- Měření a výpočty často obsahují zlomkovou část.
- Potřebujeme reprezentovat velmi malá i velmi velká čísla.
- Pevný počet bitů ale stále dovoluje jen konečný počet hodnot.

Kompromis

Čísla s plovoucí řádovou čárkou nabízejí široký rozsah za cenu omezené přesnosti.

Vědecký zápis jako inspirace

$$6,022 \times 10^{23}$$

- znaménko říká, zda je hodnota kladná nebo záporná,
- významné číslice určují přesnost,
- exponent posouvá řádovou čárku a určuje velikost.

Počítačový float používá stejnou myšlenku, ale v soustavě o základu 2.

IEEE 754: běžná reprezentace float



- **binary32**: $1 + 8 + 23$ bitů
- **binary64**: $1 + 11 + 52$ bitů
- Pythonový float je na běžných implementacích binary64.

Co jednotlivé části znamenají?

Pro běžná normalizovaná čísla zjednodušeně:

$$(-1)^{\text{znaménko}} \times 1, \text{zlomek} \times 2^{\text{posunutý exponent}}$$

- Znaménko volí kladnou nebo zápornou hodnotu.
- Exponent určuje řád neboli přibližnou velikost.
- Zlomková část nese významné binární číslice.

Pro tento kurz

Důležitější než ruční kódování bitů je rozumět omezenému rozsahu a přesnosti.

Desetinné číslo nemusí být uloženo přesně

- Konečný binární rozvoj mají například 0,5 nebo 0,25.
- Desetinná hodnota 0,1 má v binární soustavě nekonečný periodický rozvoj.
- Uloží se proto nejbližší reprezentovatelná hodnota.

Důsledek

Výsledky s float porovnáváme s vhodnou tolerancí, ne vždy přesnou rovností.

Zaokrouhlovací chyba není chyba Pythonu

```
result = 0.1 + 0.2  
print(result)           # 0.30000000000000004
```

- Operace začíná už mírně zaokrouhlenými vstupy.
- Další operace mohou chybu zvětšit nebo naopak skrýt.
- Pro peníze a přesnou desítkovou aritmetiku existují jiné typy, například `Decimal`.

Zvláštní hodnoty IEEE 754

- $+\infty$ a $-\infty$: například výsledek překročení rozsahu v některých výpočtech,
- **NaN** (Not a Number): neurčitý nebo nedefinovaný výsledek,
- $+0$ a -0 : dvě znaménkové podoby nuly,
- subnormální čísla: velmi malé hodnoty blízko nule.

NaN je zvláštní

NaN se nerovná ani samo sobě; ke kontrole používáme určené funkce.

Základní skalární typy v Pythonu

Typ	Význam	Příklad
int	celé číslo	-42
float	číslo s plovoucí čárkou	3.14
bool	pravdivostní hodnota	True
str	text	"ahoj"
NoneType	nepřítomnost hodnoty	None

Typ můžeme zjistit funkcí `type(value)`.

bool a None nejsou totéž

- bool má dvě hodnoty: True a False.
- None vyjadřuje, že hodnota není k dispozici nebo nebyla určena.
- 0, prázdný text a None mají odlišný význam.

Typ zachycuje význam

„Naměřili jsme nulu“ není totéž jako „měření chybí“.

Text: znaky nejsou bajty

- Pythonový `str` je posloupnost znaků Unicode.
- **Unicode** přiřazuje znakům kódové body: písmena, symboly i emoji.
- Při uložení nebo přenosu se text převádí na bajty pomocí **kódování**.
- Běžné kódování je UTF-8; jeden znak v něm může zabrat různý počet bajtů.

Častý omyl

Počet znaků textu nemusí být stejný jako počet bajtů v souboru.

Proměnná, hodnota a typ

- Proměnná je **jméno**, které odkazuje na objekt v paměti.
- Typ patří hodnotě (objektu), ne natrvalo jménu proměnné.
- Python určuje typ za běhu a jedno jméno může později odkazovat na jiný typ.

Dynamické typování

Nemusíme typ proměnné předem deklarovat, ale typy hodnot stále určují povolené operace.

Převod typu musí být výslovný

```
count_text = "12"  
count = int(count_text)  
average = float("3.5")  
label = str(42)
```

- `int()`, `float()` a `str()` vytvářejí hodnotu požadovaného typu.
- Ne každý převod dává smysl: `int("ahoj")` skončí chybou.
- Převod může ztrácet informaci: `int(3.9)` je 3.

Kontejner uchovává více hodnot

Kontejnerový typ

Hodnota, která sdružuje jiné hodnoty a určuje způsob přístupu k nim.

- Záleží na pořadí prvků?
- Mohou se prvky opakovat?
- Lze kontejner po vytvoření měnit?
- Přistupujeme pozicí, hodnotou, nebo klíčem?

Čtyři běžné kontejnery Pythonu

Typ	Uspořádání	Měnitelný	Typické použití
list	ano	ano	posloupnost prvků
tuple	ano	ne	pevná skupina hodnot
set	bez pozičního pořadí	ano	unikátní prvky
dict	pořadí vložení	ano	dvojice klíč–hodnota

Volba kontejneru vyjadřuje, jak s daty zamýšlíme pracovat.

list a tuple: uspořádané hodnoty

list

```
["red", "green", "blue"]
```

- lze měnit,
- přístup pomocí indexu,
- duplicity jsou povolené.

tuple

```
(1920, 1080)
```

- po vytvoření se nemění,
- přístup pomocí indexu,
- vhodná pevná struktura.

set: množina unikátních hodnot

- Prvek je v množině nejvýše jednou.
- Nepracujeme s jeho pozičním indexem.
- Množina přirozeně podporuje sjednocení, průnik a rozdíl.
- Typická otázka zní: „Je tato hodnota přítomná?“

Příklad významu

```
allowed_roles = {"student", "teacher"}
```

dict: hodnoty pojmenované klíči

- Slovník ukládá dvojice **klíč–hodnota**.
- Každý klíč je ve slovníku jedinečný.
- Hodnotu hledáme podle klíče, nikoli podle číselné pozice.

Příklad významu

```
person = {"name": "Eva", "age": 21}
```

Slovník se hodí pro záznam s pojmenovanými položkami nebo pro rychlé vyhledávání.

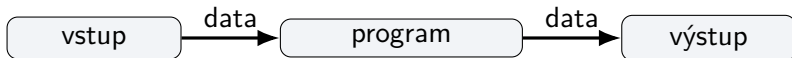
Měnitelnost ovlivňuje chování programu

- **Mutable:** objekt lze změnit na místě — například `list`, `set`, `dict`.
- **Immutable:** po vytvoření se nemění — například `int`, `float`, `str`, `tuple`.
- Dvě proměnné mohou odkazovat na tentýž měnitelný objekt.

Praktický důsledek

Změna sdíleného seznamu může být viditelná přes více různých jmen.

Program komunikuje pomocí proudů



- **Proud (stream)** je postupná posloupnost dat.
- Zdroj nebo cíl může být terminál, soubor, zařízení i jiný program.
- Program nemusí znát fyzické provedení druhého konce proudu.

Standardní vstup a výstupy

- **stdin**: standardní vstup, obvykle klávesnice nebo přesměrovaná data,
- **stdout**: běžný výstup programu,
- **stderr**: diagnostické a chybové zprávy.

Proč jsou oddělené?

Výsledek programu lze uložit nebo předat dál, zatímco chyby zůstanou viditelné uživateli.

Výstup v Pythonu: `print()`

```
name = "Eva"  
score = 42  
print("Student:", name, "body:", score)
```

- `print()` převede hodnoty na text a zapíše je na standardní výstup.
- Více argumentů standardně oddělí mezerou.
- Na konec standardně přidá znak nového řádku.

Vstup v Pythonu: `input()`

```
age_text = input("Zadejte vek: ")  
age = int(age_text)
```

- `input()` zobrazí výzvu a čeká na řádek standardního vstupu.
- Výsledkem je vždy `str`, i když uživatel napsal číslice.
- Význam vstupu určíme až kontrolou a případným převodem typu.

Rozhraní programu je dohoda

- Program musí vědět, v jakém formátu vstup očekává.
- Uživatel nebo jiný program musí vědět, co výstup znamená.
- Typ, jednotka, pořadí i kódování jsou součástí této dohody.

„Zadejte číslo“ často nestačí

Celé, nebo desetinné? V jakém rozsahu? S desetinnou čárkou, nebo tečkou? V jaké jednotce?

Co si zapamatovat?

- Bity získávají význam teprve díky zvolené reprezentaci a datovému typu.
- Signed a unsigned celá čísla rozdělují stejný počet bitů jinak.
- IEEE 754 nabízí velký rozsah, ale mnoho hodnot ukládá pouze přibližně.
- Python má skalární typy i kontejnery s různými vlastnostmi.
- `input()` vrací text; převod na číslo je samostatný krok.
- Standardní proudy oddělují vstup, běžný výstup a chybové zprávy.

Otázky?